



CS240 Algorithm Design and Analysis

Lecture 12

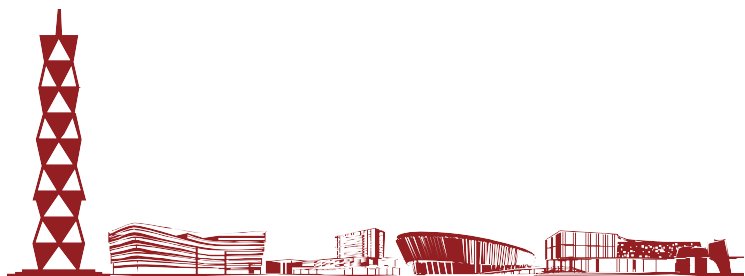
NP and Computational Intractability (Cont.)

Qingying Hao

Spring 2026
2026.04.08



Co-NP and the Asymmetry of NP

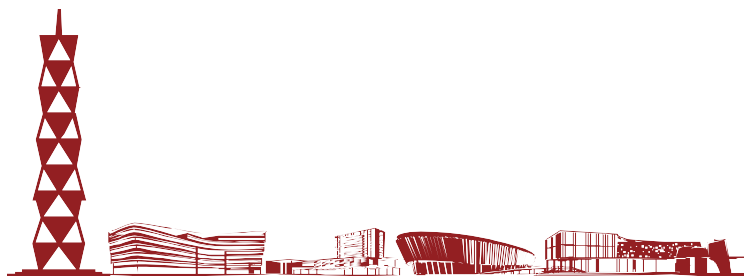




Asymmetry of NP



- **NP.** Decision problems for which there is a poly-time certifier
- **Ex** SAT, HAM-CYCLE, COMPOSITES
- **Asymmetry of NP.** We only need to have short proofs of yes instances
- It may not be easy to show proofs of no instances
- **Ex1.** SAT vs. UN-SAT
 - Can prove a CNF formula is satisfiable by giving such an assignment
 - How could we prove that a formula is **not** satisfiable?
- SAT = satisfiable
- UN-SAT = unsatisfiable





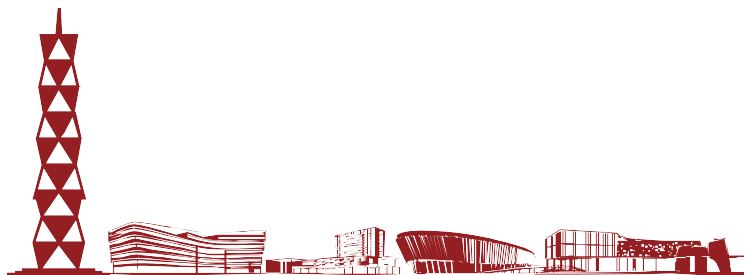
Asymmetry of NP



- **Asymmetry of NP.** We only need to have short proofs of yes instances
- **Ex2.** HAM-CYCLE vs. NO-HAM-CYCLE
 - Can prove a graph is Hamiltonian by giving such a Hamiltonian cycle
 - How could we prove that a graph is not Hamiltonian?

Q. How to classify UN-SAT and NO-HAMILTON-CYCLE ?

- $\text{SAT} \in \text{NP-complete}$ and $\text{SAT} \equiv_P \text{UN-SAT}$.
- $\text{HAMILTON-CYCLE} \in \text{NP-complete}$ and $\text{HAMILTON-CYCLE} \equiv_P \text{NO-HAMILTON-CYCLE}$.
- But neither UN-SAT nor NO-HAMILTON-CYCLE are known to be in NP.





SAT, UNSAT, and TAUTOLOGY



- **SAT:** A formula is **satisfiable** if there exists **some** assignment that makes it true.
$$\exists x \phi(x) = 1$$
- **UNSAT:** A formula is **unsatisfiable** if there is **no** assignment that makes it true.
$$\forall x \phi(x) = 0$$
- **TAUTOLOGY:** A formula is a **tautology** if **every** assignment makes it true.
$$\forall x \phi(x) = 1 \quad (\text{Ex. } x \vee \neg x)$$
- ϕ is UNSAT $\Leftrightarrow \neg\phi$ is TAUTOLOGY

SAT:

$$x \wedge y$$

(satisfiable, e.g. $x = 1, y = 1$)

UNSAT:

$$x \wedge \neg x$$

(always false)

TAUTOLOGY:

$$x \vee \neg x$$

(always true)





NP and co-NP



- **NP.** Decision problem for which there is a poly-time certifier
- **Ex.** SAT, HAM-CYCLE, COMPOSITES
- **Def.** Given a decision problem X , its complement $\overline{X}$ is the same problem with the yes and no answers reverse
- **Ex.** $\overline{X} = \{0, 1, 4, 6, 8, 9, 10, 12, 14, 15, \dots\}$
 $X = \{2, 3, 5, 7, 11, 13, 17, 23, 29, \dots\}$
- **Co-NP.** Complements of decision problems in NP
- For no instance, there is a succinct disqualifier
- **Ex.** TAUTOLOGY, NO-HAM-CYCLE, PRIMES

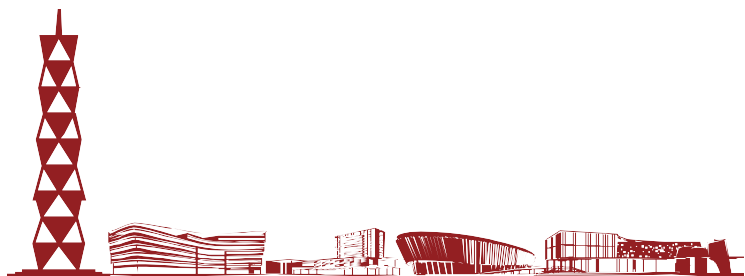




NP = co-NP?



- **Fundamental question.** Does $NP = co-NP$?
 - Do yes instances have succinct certificates iff no instances do?
 - Consensus opinion: no
- **Theorem.** If $NP \neq co-NP$, then $P \neq NP$
- **Pf idea**
 - P is closed under complementation
 - If $P = NP$, then NP is closed under complementation
 - In other words, $NP = co-NP$
 - This is the contrapositive of the theorem

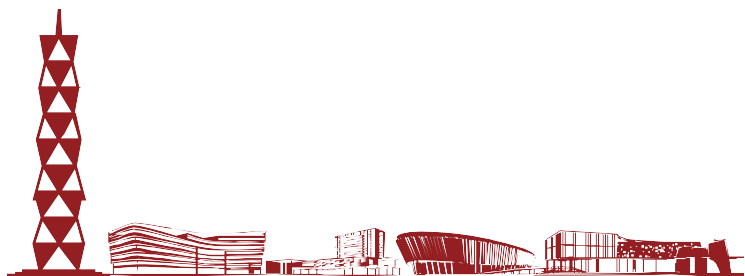




Good Characterizations



- **Good characterization.** [Edmonds 1965] $\text{NP} \cap \text{co-NP}$
 - If problem X is in both NP and co-NP , then:
 - For yes instance, there is a succinct certificate
 - For no instance, there is a succinct disqualifier
 - Provides conceptual leverage for reasoning about a problem
- **Ex.** Given a bipartite graph, is there a perfect matching
 - If yes, can exhibit a perfect matching
 - If no, can exhibit a set of nodes S such that $|N(S)| < |S|$





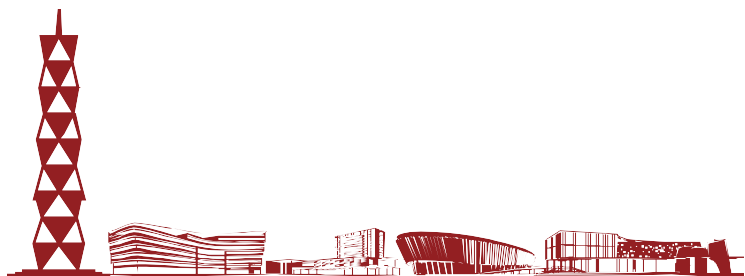
Good Characterizations



- **Observation.** $P \subseteq NP \cap \text{co-NP}$
- **Fundamental open question.** Does $P = NP \cap \text{co-NP}$?
 - Mixed opinions
 - Many examples where problem found to have a non-trivial good characterization, but only years later discovered to be in P
 - Linear programming [Khachiyan, 1979]
 - Primality testing [Agrawal-Kayal-Saxena, 2002]
- **Fact.** Factoring is in $NP \cap \text{co-NP}$, but not known to be in P



If poly-time algorithm for factoring, can break RSA cryptosystem



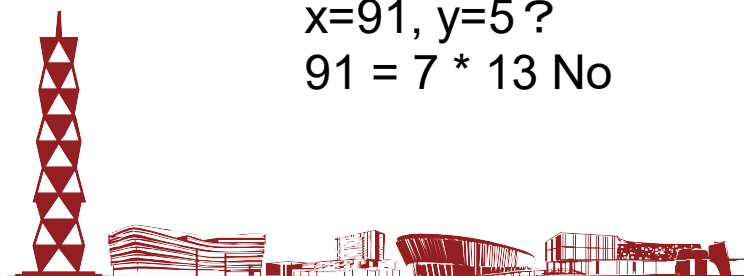


FACTOR is in $NP \cap co-NP$



- **FACTORIZE**. Given an integer x , find its prime factorization
- **FACTOR**. Given two integers x and y , does x have a nontrivial factor less than y ?
 $x=91, y=10$
 $91=7 \cdot 13$
 $7 < 10$, yes
- **Theorem**. $FACTOR \equiv_p FACTORIZE$.
- **Pf**. Binary search to find a factor, divide the factor and repeat
 $x=91, y=5$?
- **Theorem**. $FACTOR$ is in $NP \cap co-NP$
- **Pf**.
 - Certificate: a factor p of x that is less than y
 - Disqualifier: the prime factorization of x (where each prime factor is $\geq y$), along with a Pratt's certificate that each factor is prime

$x=91, y=5$?
 $91 = 7 * 13$ No

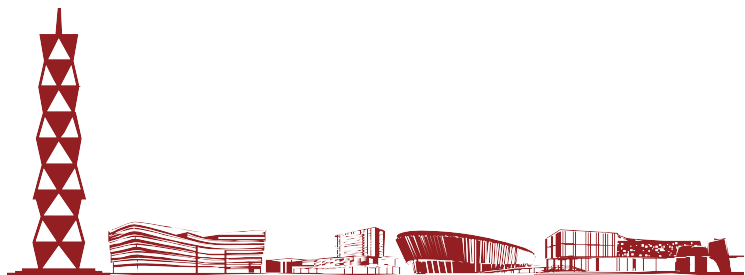




Primality Testing and Factoring

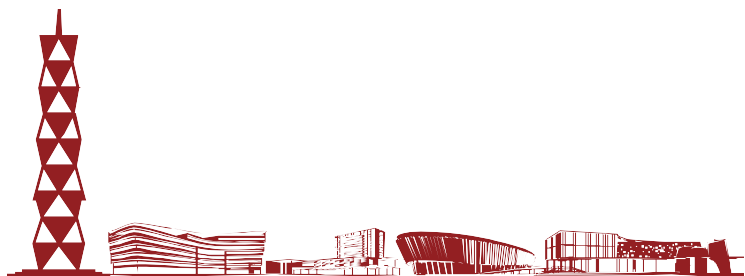


- **Q: Is PRIMES in P?**
 - Yes! (Agrawal-Kayal-Saxena, 2002)
- **Q: Is FACTOR in P?**
 - Consensus: No
- **RSA cryptosystem**
 - Based on dichotomy between complexity of two problems
 - To use RSA, must generate large primes efficiently
 - To break RSA, must find efficient factoring algorithm



- **Basic genres**

- Packing problems: SET-PACKING, INDEPENDENT SET
- Covering problems: SET-COVER, VERTEX-COVER
- Constraint satisfaction problems: SAT, 3-SAT
- **Sequencing problems: HAMILTONIAN-CYCLE, TSP**
- Partitioning problems: 3D-MATCHING, 3-COLOR
- Numerical problems: SUBSET-SUM, KNAPSACK

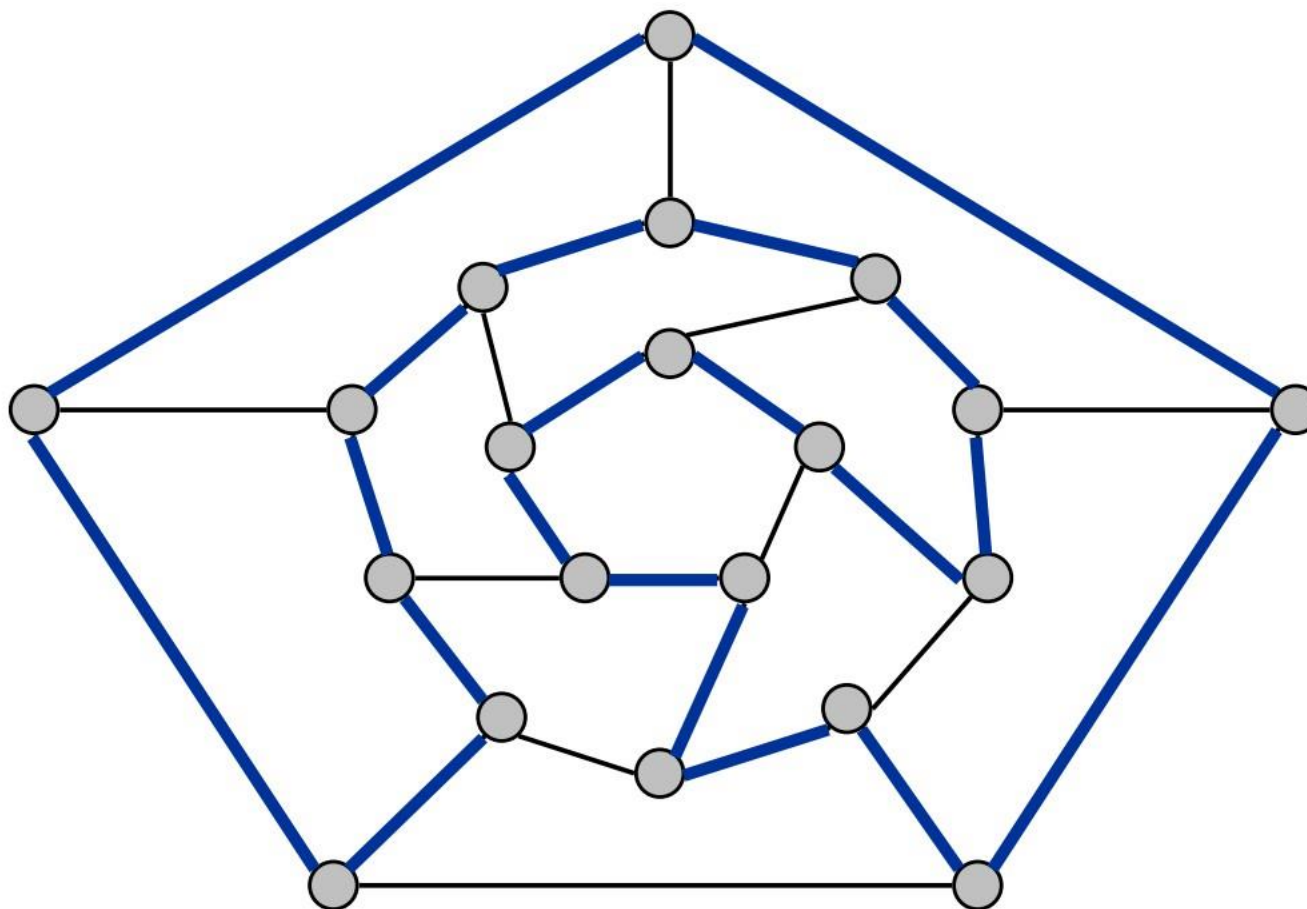




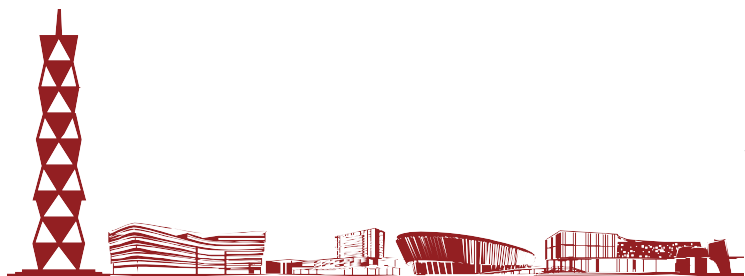
Hamiltonian Cycle



- **HAM-CYCLE** given an undirected graph $G = (V, E)$, does there exist a simple cycle that contains every node in V



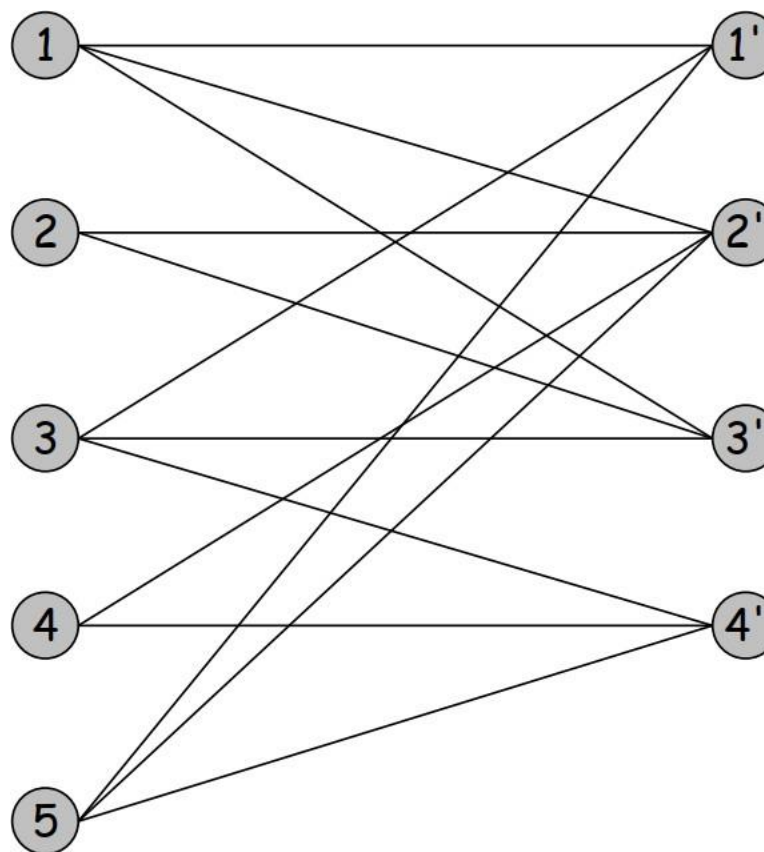
YES: vertices and faces of a dodecahedron



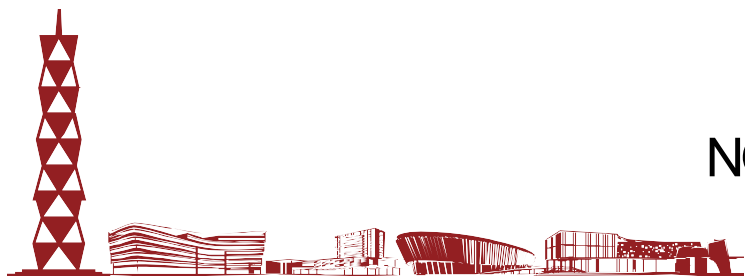
Hamiltonian Cycle



- **HAM-CYCLE** given an undirected graph $G = (V, E)$, does there exist a simple cycle that contains every node in V



NO: bipartite graph with odd number of nodes

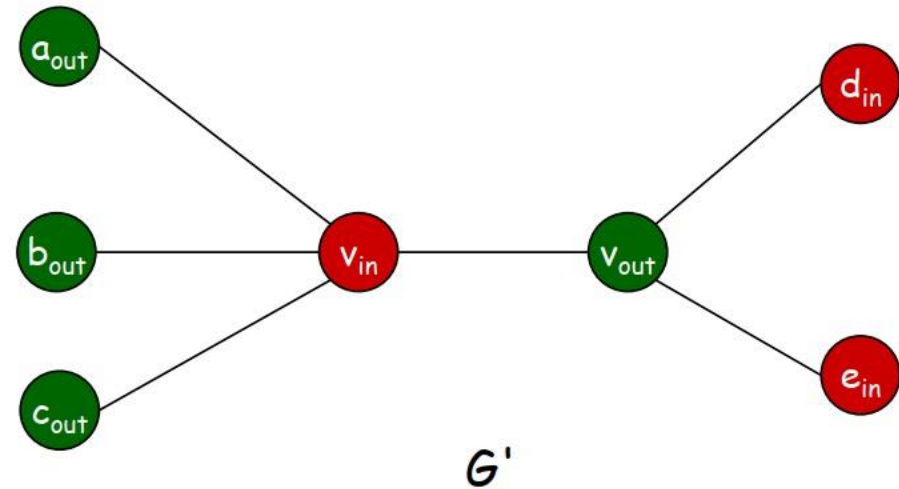
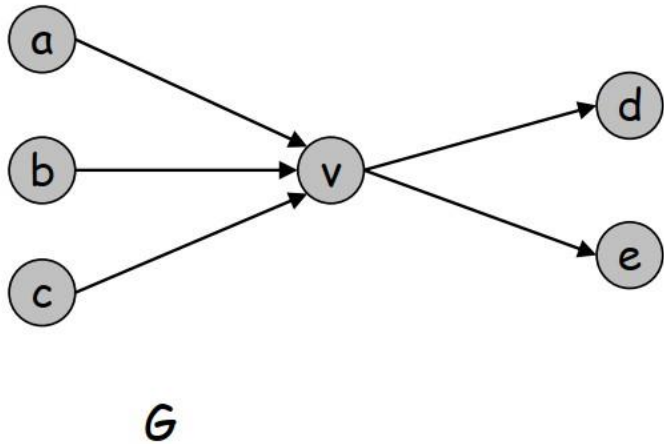




Directed Hamiltonian Cycle



- **DIR-HAM-CYCLE**: given a **digraph** $G = (V, E)$, does there exist a simple directed cycle that contains every node in V ?
- **Claim**. $\text{DIR-HAM-CYCLE} \leq_p \text{HAM-CYCLE}$
- **Pf**. Given a directed graph $G = (V, E)$, construct an undirected graph G'





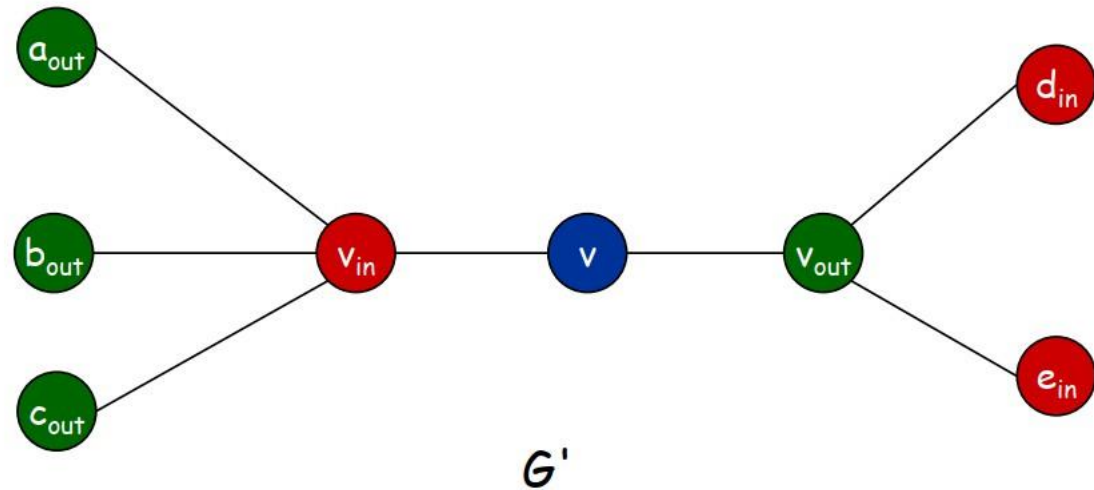
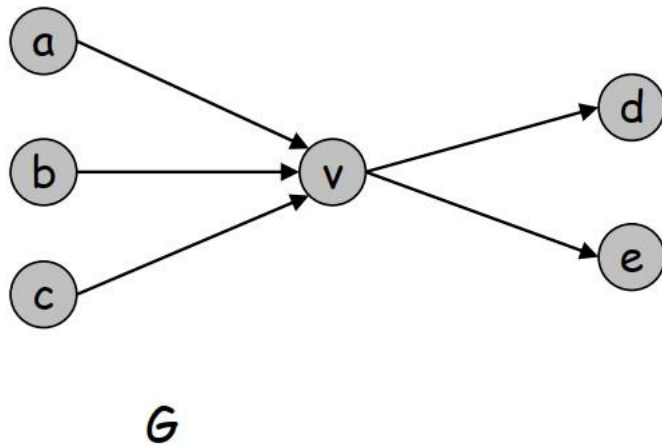
Directed Hamiltonian Cycle



- **DIR-HAM-CYCLE**: given a **digraph** $G = (V, E)$, does there exist a simple directed cycle that contains every node in V ?
- **Claim**. $\text{DIR-HAM-CYCLE} \leq_p \text{HAM-CYCLE}$
- **Pf**. Given a directed graph $G = (V, E)$, construct an undirected graph G' with $3n$ nodes

• **Recipe to establish NP-completeness of problem Y**

- Step 1. Show that Y is in NP
- Step 2. Choose an NP-complete problem X
- Step 3. Prove that $X \leq_p Y$

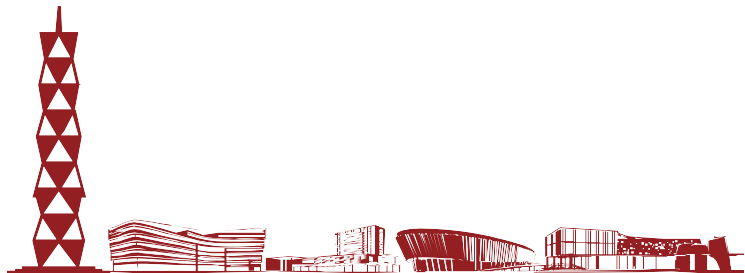




Directed Hamiltonian Cycle



- **Claim.** G has a Hamiltonian cycle iff G' does
- **Pf. $\rightarrow$**
 - Suppose G has a directed Hamiltonian cycle
 - Then G' has an undirected Hamiltonian cycle (same order)
- **Pf. $\leftarrow$**
 - Suppose G' has an undirected Hamiltonian cycle
 - The cycle must visit nodes in G' using one of following two orders:
 - $\dots, B, G, R, B, G, R, B, G, R, B, \dots$
 - $\dots, B, R, G, B, R, G, B, R, G, B, \dots$
 - Blue nodes in this cycle make up directed Hamiltonian cycle in G , or reverse of one

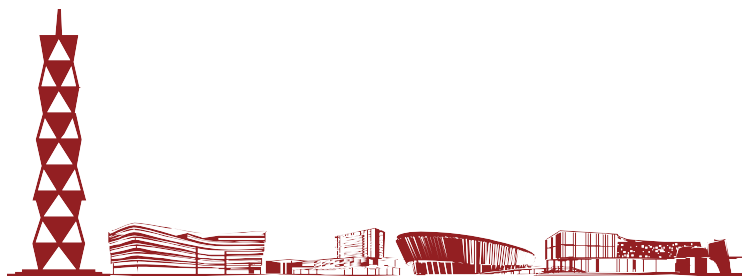




3-SAT Reduces to Directed Hamiltonian Cycle



- **Claim.** $3\text{-SAT} \leq_p \text{DIR-HAM-CYCLE}$
- **Pf.** Given an instance ϕ of 3-SAT, we construct an instance of DIR-HAM-CYCLE that has a Hamiltonian cycle iff ϕ is satisfiable

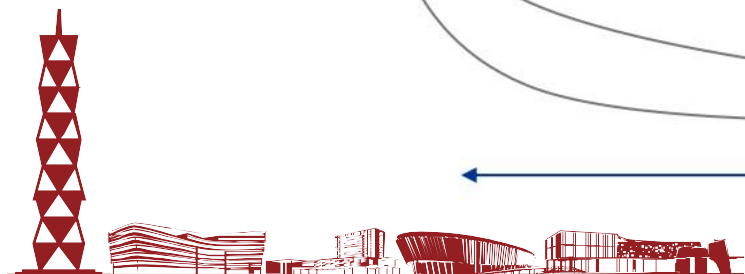
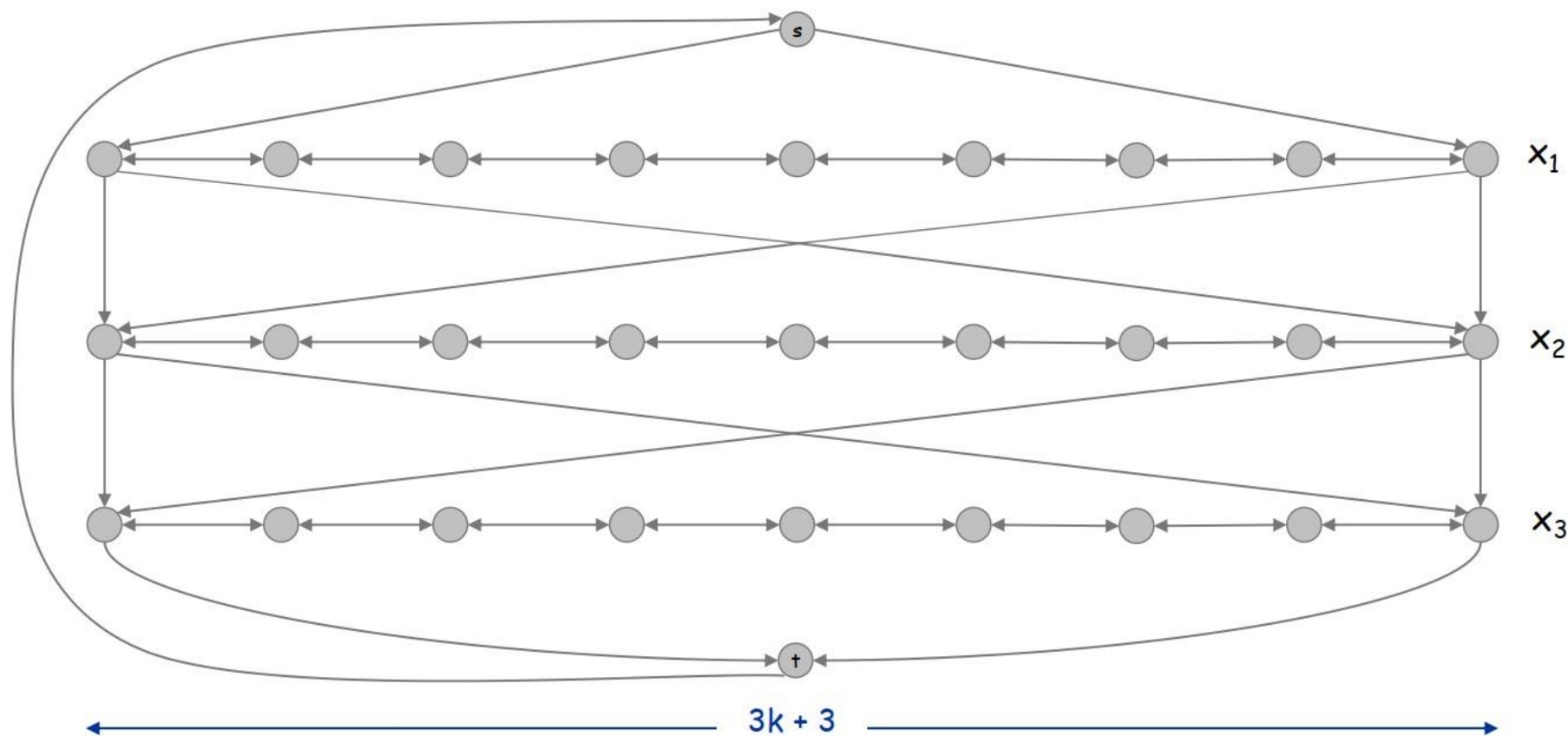




3-SAT Reduces to Directed Hamiltonian Cycle



- **Construction.** Given 3-SAT instance ϕ with n variable x_i and k clauses.
 - Construct G to have 2^n Hamiltonian cycles
 - Intuition: traverse path i from left to right $\leftrightarrow$ set variable $x_i = 1$

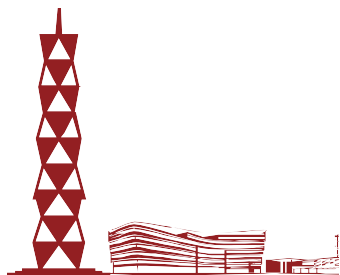
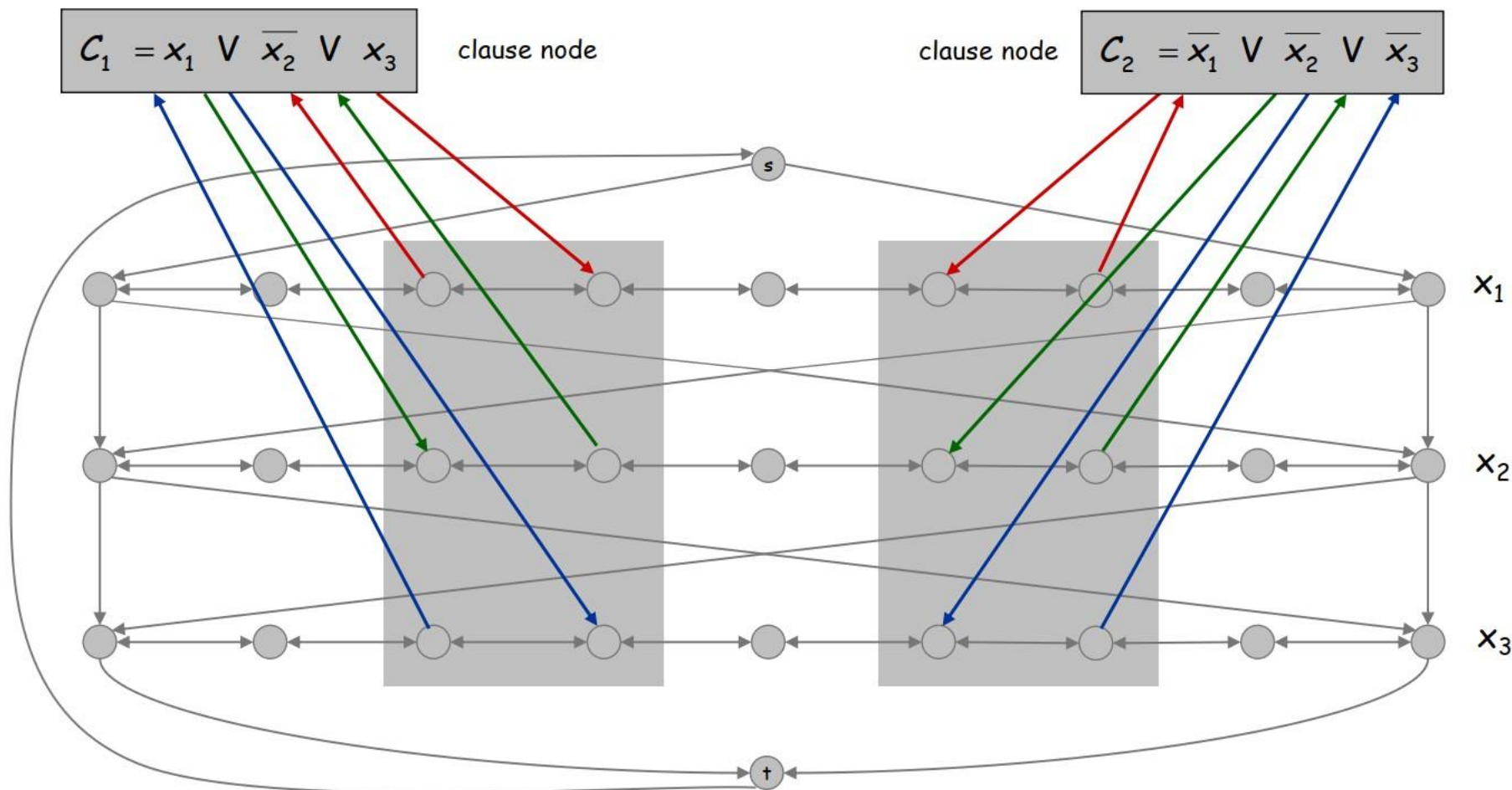




3-SAT Reduces to Directed Hamiltonian Cycle



- **Construction.** Given 3-SAT instance ϕ with n variable x_i and k clauses.
 - For each clause: add a node and 6 edges

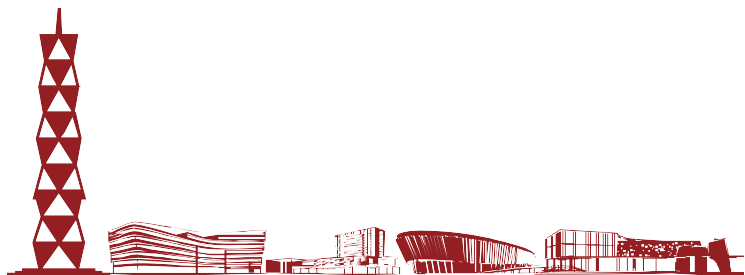




3-SAT Reduces to Directed Hamiltonian Cycle



- **Claim.** ϕ is satisfiable iff G has a Hamiltonian cycle
- **Pf.** $\rightarrow$
 - Suppose 3-SAT instance has satisfying assignment x^*
 - Then, define Hamiltonian cycle in G as follows:
 - If $x_i^* = 1$, traverse row i from left to right
 - If $x_i^* = 0$, traverse row i from right to left
 - For each clause C_j , there will be at least one row i in which we are going in “correct” direction to splice node C_j into tour

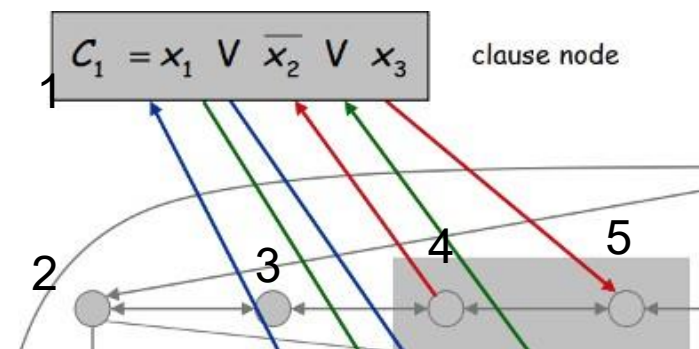




3-SAT Reduces to Directed Hamiltonian Cycle



- **Claim.** ϕ is satisfiable iff G has a Hamiltonian cycle
- **Pf.** $\leftarrow$
 - Suppose G has a Hamiltonian cycle T
 - If the cycle T enters clause node C_j , it must depart on mate edge
 - Thus, nodes immediately before and after C_j are connected by an edge e in G
 - Removing C_j from cycle, and replacing it with edge e yields Hamiltonian cycle on $G - \{C_j\}$
 - Continuing in this way, we are left with Hamiltonian cycle T' in $G - \{C_1, C_2, \dots, C_k\}$
 - Set $x_i^* = 1$ iff T' traverse row i left to right
 - Since T' visits each clause node C_j , at least one of the paths is traversed in “correct” direction, and each clause is satisfied





Next Time: NP and Computational Intractability (Cont.)

